the sorted array to a single element (the last one tested). When it comes to the value comparison step, only the last element in the first array is compared to elements of the comparison array(s). In our case, the last element in `$advertised` had the registration value of `'BBB 1B'`, it is the only value tested against `$outForRepair`, and the intersect is empty.

To fix the sorting step, we need a little more logic in the value comparison callback function:

```
$deListUntilRepaired = array_uintersect($advertised, $outForRepair,
function ($ads, $reps) {
    if (isset($reps['x_registration'])) {
        return $ads['registration'] <=> $reps['x_registration'];
    } else {
        return $ads['registration'] <=> $reps['registration'];
    }
});
print_r($deListUntilRepaired);
```

```
Output:
Array
(
    [0] => Array
        (
            [registration] => AAA 1A
            [model] => Ford Prefect
        )

)
```

By recognizing that the comparison callback is also a sorting callback (and compares values within arrays as well as between), we can write it in such a way that differences in sub-array keys can be managed however we wish (and if we'd used objects, we could account for different property names or method calls in the same way).

Remember when we looked at `array_change_key_case()`, I promised there was a way to handle keys that had a greater difference than just case? Let's revisit that example, but instead of changing the case of every key, we'll use a key comparison callback function: